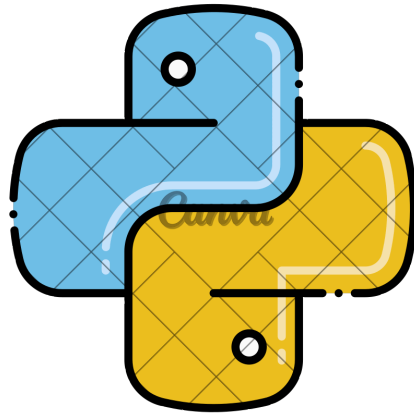




Understanding `__init__.py` in Python

One file. Big impact on imports, packages, and architecture.

Why this matters

- Frequently asked in Python interviews (Capgemini, TCS, Cognizant)
- Misunderstood by many developers
- Critical for clean package design

A R U N A R U N I S T O

The Problem It Solves

Before `__init__.py`

Python originally had no way to know if a directory was a package.

```
project/  
  utils/  
    helpers.py
```

✗ Python didn't treat `utils` as a package.

The Solution

Adding a special file:

```
utils/  
  __init__.py  
  helpers.py
```

✓ Now Python recognizes it as a package

What `__init__.py` Actually Is

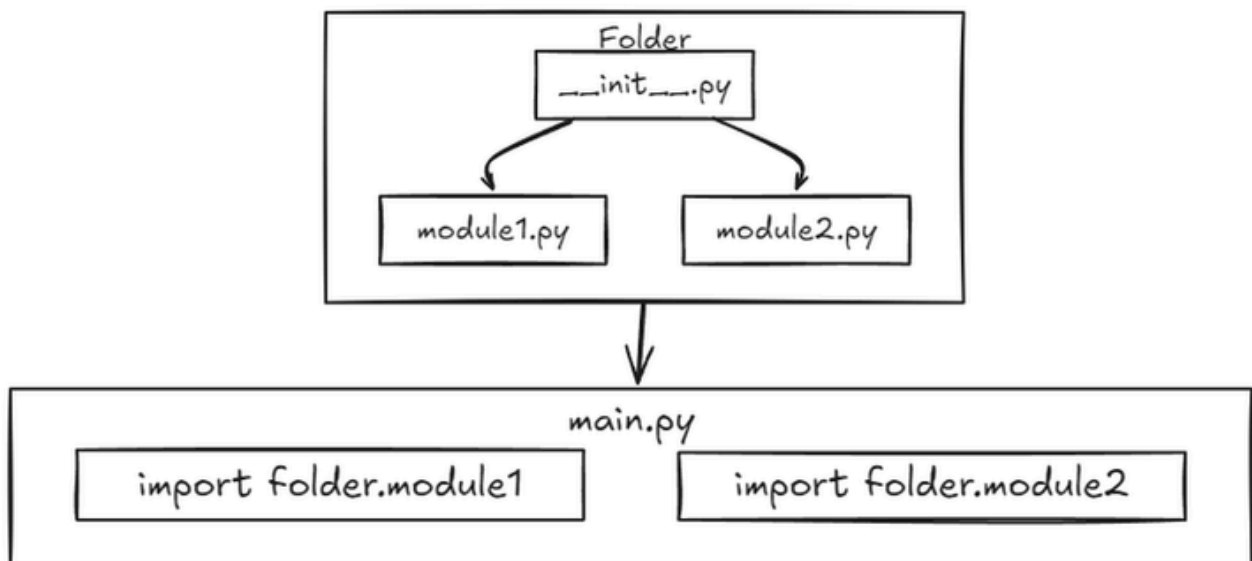
Straight facts

- `__init__.py` is a normal Python file
- It is executed once when the package is imported
- It runs before accessing any submodules

import mypackage

➔ Python executes:

mypackage/__init__.py



Common Misconceptions (IMPORTANT)

✗ Wrong assumptions many candidates make

- `__init__.py` ✗ is NOT a constructor
- `__init__.py` ✗ is NOT related to classes
- `__init__.py` ✗ does NOT run on every import
- `__init__.py` ✗ is NOT the same as `__init__()`

✓ Reality

<code>__init__.py</code>	<code>__init__()</code>
Package-level	Class-level
Runs once	Runs per object
Controls imports	Initializes objects

Is `__init__.py` Mandatory Today?

Python 3.3+ → Namespace Packages

✓ Import works without `__init__.py`

But in real projects...

- Django → still expects it
- Explicit imports > implicit behavior
- Used to control initialization & exports

✓ Best practice: still use `__init__.py`

Interview Trap Question 🚨

Question:

Is `__init__.py` required in Python 3?

Weak answer ✗

“No, it’s not required.”

Strong answer ✓

“Technically optional due to namespace packages, but in real-world projects we still use it to control imports, initialization, and package APIs.”

This answer scores points.

Final Takeaways

- `__init__.py` defines a package's behavior
- It executes when the package is imported
- Used to:
 - Initialize packages
 - Simplify imports
 - Control public APIs
- Still widely used in production systems

Understanding imports = understanding
Python architecture